TagAlert App (TagAlert.me): Capstone Project Final Report

Elizabeth Hadik-Barkoczy

College of Applied Science and Technology, University of Arizona

APCV 498: Capstone Project

Dr. Betül Czerkawski

Fall 2024 (Incomplete; work completed November 2026)

November 9, 2026

TagAlert App (TagAlert.me): Capstone Project Final Report

Executive Summary

TagAlert is an iPhone app that looks for Apple AirTags near the user and warns when one appears to be traveling with them. The project began in Fall 2024. A December 2025 version of the prototype, built to test basic functionality, used simulated data and static numbers. The app was then rebuilt and tested on a physical iPhone between November 1 and November 9, 2026. The original Fall 2024 test logs were lost, so every result in this report comes from the November 2026 test logs, which are listed in Appendix C.

The central technical finding is that iOS does not give third-party apps the Find My advertisement that identifies an AirTag. In every scan recorded, nearby AirTags appeared only as unnamed Bluetooth devices with no identifying data. TagAlert confirms an AirTag by briefly connecting to such a device and checking for the AirTag's Bluetooth service, a method first published by the AirGuard project at TU Darmstadt (Heinrich et al., 2022). TagAlert's code is an independent implementation.

The final build (Version 6 build b) confirmed AirTags 3 to 28 seconds after scanning began in the 21 confirmations recorded after the probe queue was reordered (median 10.4 s). In the final walking test, with a tag in the user's pocket, it raised Medium after 72 seconds and 77 meters and High after 123 seconds and 148 meters of walking, then offered to make the tag beep. In seven stationary control tests with three or four AirTags near the phone, the longest lasting 47 minutes, it raised no alerts, because the final threat rules require a tag to move with the user. Two false alerts on early walking tests were traced to specific rule defects, fixed, and did not recur on the next four walks. The app can also make a tag beep so it can be found: 26 of 27 commands

sent to the user's own tags while separated from their owner were accepted, and a controlled test showed Apple blocks the beep when the tag's owner is nearby.

The testing is small: one phone, one home, one walking route, and three to four AirTags over nine days. TagAlert works only while the app is open, and in a 47-minute run with the screen on it used about 17% of the battery per hour. These limits are stated with the results.

Problem Statement and Background

The AirTag Technology

Apple AirTags are coin-sized Bluetooth Low Energy (BLE) trackers. An AirTag broadcasts a short advertisement that nearby Apple devices pick up and relay, encrypted, to the owner through Apple's Find My network. The advertisement uses keys that rotate over time, so the identifier a listener sees changes periodically (Heinrich et al., 2021). AirTags are designed to find lost keys and bags, but their small size and the reach of the Find My network also make them usable for following a person without consent.

The Security Problem

A person can hide an AirTag in someone else's bag or car and follow their location in the Find My app. Apple added protections: iPhones can warn their owners about an unknown AirTag that travels with them, and an AirTag separated from its owner plays a sound. In June 2021 Apple shortened the time before that sound from three days to a random time between 8 and 24 hours (MacTech, 2021). A person being tracked may therefore be followed for hours before any sound plays, and once warned may still struggle to find the tag.

Research and Need Identification

AirGuard, an open-source app from the Secure Mobile Networking Lab at TU Darmstadt, showed that a third-party app could detect Find My trackers and play their sound, first on

Android and later on iOS (Heinrich et al., 2022). This project set out to build a simple iPhone app for non-technical users that detects nearby AirTags, judges whether one is following the user rather than passing by, and helps the user find it. A secondary goal, set during the November 2026 re-test, was to measure every claim the app makes rather than estimate it.

Objectives and Research Questions

Primary Objective

The primary objective, from the approved proposal, was to create a user-friendly smartphone app that lets a user detect, and act on, unapproved AirTags in their vicinity, while balancing detection, battery use and ease of use.

Research Questions

The proposal posed four questions. A fifth, on threat assessment, was added during development.

1. **Battery Optimization:** How can the app limit its drain on the phone's battery without sacrificing detection?
2. **User Experience Design:** How can the app be simple enough for non-technical users?
3. **Multiple Device Detection:** How does the app respond when it detects more than one AirTag at once?
4. **False Positive Reduction:** How can the app tell an AirTag that is passing by from one that is traveling with the user?
5. **Threat Assessment:** What criteria should set the warning level shown to the user?

Project Goals

1. **Efficient Algorithm Performance:** scan with as little radio time as practical.

2. **Intuitive User Experience:** give clear, plain-language feedback that needs no technical knowledge.
3. **Compatibility:** test on real iOS hardware.
4. **Actionable Response:** give the user something to do when a tag is found: locate it and get safety guidance.

Project History and Evidence Standard

The project was built in stages, each with a different purpose.

- **Fall 2024:** research, design and an initial Swift prototype. The test logs from this period were lost and cannot be used as evidence.
- **December 2025 functional prototype:** a version built to test basic functionality: the interface, navigation, threat-level display and alert flow. It deliberately used simulated tracker data and static numbers so that each screen could be exercised without hardware. The static figures shown in that prototype and in early drafts (for example 92% accuracy and under 2% false positives) were placeholders. This report replaces them with measured results, and features from that stage that were not carried into the final build (map directions, disabling a tag, background scanning, a Critical level) are not described as implemented.
- **November 1 to 9, 2026, live build and field testing:** the app was rebuilt to scan live, with a comma-separated values (CSV) log that records every scan, connection attempt, threat change, notification and beep command. It was tested on an iPhone SE (2nd generation) running iOS 17.6.1 with the user's own AirTags. Each code change bumped a build letter (5a to 5z, then 6a and 6b), and every log records the build that produced it.

Evidence standard: a number appears in this report only if it can be recomputed from a test log. The one exception is the phone's status-bar battery reading, which is recorded with the battery log (Battery, below).

Methodology

Development Approach

Development followed the iterative "wash, rinse and repeat" approach in the proposal. In November 2026 each cycle was: run a test with a written protocol and an on-screen marker in the log (for example "B3 Start"), analyze the log, identify the root cause of any wrong result, change one rule, and retest. Twenty-three findings were recorded this way; the rule changes they produced are listed in Appendix B.

Phases

1. **Research (Fall 2024, revisited November 2026):** BLE scanning, Apple's Find My protocol, and existing detectors. In November 2026 this included reading AirGuard's published source to understand how it detects AirTags on iOS.
2. **Algorithm and interface design (Fall 2024):** scan schedule, threat levels, and the main screen layout.
3. **Core development (Fall 2024 prototype; rebuilt November 2026):** live scanning, AirTag confirmation, threat rules, movement check, notifications, beep, settings, and the CSV test log.
4. **Testing and iteration (November 1 to 9, 2026):** stationary control tests (A series), walking tests (B series), and beep tests, with rule changes between runs.
5. **Documentation (November 2026):** this report, presentation, poster, source repository and project website.

Test Setup

- **Phone:** iPhone SE (2nd generation), iOS 17.6.1, app running in the foreground with the screen on.
- **Tags:** the user's AirTags, identified in logs by shortened identifiers DEADE924, CC09DDBA, 8825D224 and 4CDEDA2F. AirTags rotate their Bluetooth address, so iOS may give the same physical tag a new identifier after a rotation.
- **Owner separation:** for beep tests the phone that owns the tags had Bluetooth turned off, so the tags behaved as if separated from their owner. This matches the threat model of a tag planted by someone else.
- **Stationary tests (A3 to A8, BAT1):** tags on a table beside the phone (one tag in another room in A4 to A6), phone not moved, 0.6 to 47 minutes.
- **Walking tests (B1 to B7):** one of the user's tags in their pocket, phone in hand, the same residential loop route that starts and ends at home, 2 to 7 minutes. Distances come from the phone's step counter. No Global Positioning System (GPS) reference was recorded, so distances are estimates.
- **Analysis:** each log was analyzed from the session start time. Times in this report are seconds after the user pressed Start Scanning, unless stated otherwise.

Technical Implementation Details

Detecting an AirTag on iOS

The original design looked for Apple's Find My advertisement (manufacturer data beginning 4C 00 12 19). In the first November test, with an AirTag about one meter from the phone, the app logged 33 to 44 Bluetooth devices per scan and decoded manufacturer data from Microsoft, Samsung, Garmin, Sony and Sonos devices, but recorded zero Apple manufacturer

data. This held across every later log. iOS does not pass Apple's Find My payload to third-party apps.

An AirTag instead appears as a bare device: connectable, with no name, no manufacturer data, no service list and no service data. TagAlert therefore checks each such device directly, in four steps:

1. **Queue:** every bare device seen in a scan joins the probe queue (see Probe Queue, below).
2. **Connect:** TagAlert opens a brief connection, allowing 6 s for a strong device and 10 s for a weak one.
3. **Ask for one service:** it requests only the service with universally unique identifier (UUID) 7DFC9000-7D1C-4951-86AA-8D9728F8D66C, rather than the device's full service list, so the check takes about 1 to 4 seconds.
4. **Decide and disconnect:** if the device returns that service it is confirmed as an AirTag; otherwise it is ignored. TagAlert disconnects at once and never pairs with or changes the device.

A device is confirmed as an AirTag if it offers the service with UUID 7DFC9000-7D1C-4951-86AA-8D9728F8D66C, identified by AirGuard's reverse engineering; the same service carries the sound command (Make This Tag Beep, below). A service UUID is the 128-bit identifier a Bluetooth device publishes for each feature it offers. Standard features use short shared identifiers, while a full 128-bit custom identifier belongs to one vendor's design, so unrelated devices do not offer it by chance. In the first detection test (November 2), 18 bare devices were checked: 3 offered the service, 11 did not, and 4 timed out. Each of the user's own tags confirmed this way later beeped on command, which only a device with the AirTag's sound characteristic can do. Apple does not publish this identifier, so other Find My accessories, or a

device built to imitate an AirTag, could in principle offer it too. This method follows AirGuard (Heinrich et al., 2022; Secure Mobile Networking Lab, n.d.). TagAlert's code is an independent implementation; no AirGuard code was copied.

An earlier attempt to filter scans by Apple's Find My service (FD44) returned cached, unrelated devices within milliseconds of the scan starting. These were false detections and produced false Medium notifications; the filter was removed. This is recorded as a false-positive finding rather than discarded.

Probe Queue

Connecting takes time, and a busy street can produce well over a hundred bare devices. The probe queue therefore went through several revisions, each driven by a test result:

- **Strongest signal first (5o):** in walk B1 the queue was first-in, first-out. The tag in the user's pocket was seen 1.2 s after scanning began but waited behind ten weaker, distant devices and was not confirmed until 94 s. Probing the strongest device first cut confirmation to 6.0 to 8.9 s on later walks.
- **Queue-age rule and retries (5w):** strongest-first could starve a weak tag in another room. A device that has waited 45 s now goes next regardless of signal, and a device that times out is retried later instead of being written off for the session.
- **Back-off and timeouts (5x):** retries back off at 10, 20 and 40 s, then every 60 s. Signal strength is measured in decibel-milliwatts (dBm); values closer to zero are stronger. Strong devices get 6 s to connect and weak ones (-85 dBm or below) get 10 s.
- **Show results immediately (6a):** probing starts about 1 s into a scan instead of after the 5-second listening window, and a confirmed tag appears on screen at once instead of at

the end of the scan cycle. In A7 the first tag was confirmed at 11 s but shown at 20 s; in A8 tags appeared on screen at 10.1, 11.6 and 13.1 s.

Threat Assessment

Each confirmed AirTag gets one of three levels. Time is counted from the tag's first sighting, not from its confirmation.

Level	Rule (build 6b)
Low	Seen for under 1 minute, or not moving with the user
Medium	Seen for 1 to 2 minutes and moving with the user
High	Seen for 2 minutes or more and moving with the user

Moving with the user means all of the following: the user walked at least 50 m, or spent at least 60 s in a vehicle, between the tag's first and latest sighting; the tag was seen in at least 60% of scan cycles over at least 4 cycles; and the tag was seen within the last 25 s. If the tag is unseen for 45 s the movement window starts over. Signal strength does not affect the level (see Discussion). If the phone cannot provide movement data, levels follow time only and High is capped at Medium, so the app never shows its strongest warning without evidence of movement.

Movement Check

Movement comes from Core Motion: the step counter and the phone's activity type (walking, in a vehicle). TagAlert never uses location. If the phone-wide Fitness Tracking switch is off, iOS refuses step data to every app without showing a prompt, and apps cannot open that setting. TagAlert detects this, explains it with a prompt and a banner, and falls back to time-only levels. A permission-free alternative (counting steps from the raw accelerometer) was considered and rejected because it would work around a privacy choice the user had made.

Scan Schedule

Each scan cycle listens for 5 seconds, then pauses. The default interval is 15 seconds; with Adaptive Scanning on it shortens to 10 seconds once a tag is found and lengthens when the battery is low. The user can set the base interval from 10 to 60 seconds.

Locating a Tag: Make This Tag Beep

TagAlert can ask a confirmed AirTag to play its sound by writing the command byte AF to the AirTag's sound characteristic (7DFC9001, the same command AirGuard uses). Apple's tags accept this only from a non-owner when the tag is separated from its owner. When a tag first reaches High, TagAlert asks once, "Find This Tag?", with Beep Now and Not Now, and warns that the sound could tell a nearby planter that the tag was found (Figure 2). Automatic beeping is available in Settings but is off by default, for the safety reasons given under Privacy and Ethical Considerations.

Notifications and Logging

TagAlert sends one notification per tag per level, honors the user's level settings, and shows banners while the app is open. It uses four alert sounds made for the project, with Radar Pulse as the default. The CSV test log records every event with a timestamp, the build, the battery level and whether the data was live or simulated demo data. Demo data is off by default, labeled on screen, and tagged DEMO in the log; no demo data was used for any result in this report.

Technologies Employed

Development Platforms and Languages

- **Swift and SwiftUI:** all app code and the user interface.
- **Xcode:** building and installing on the test iPhone.

- **Core Bluetooth:** scanning, connecting to bare devices, service discovery, and the beep command (Apple Inc., n.d.-a).
- **Core Motion:** step counter and activity type for the movement check (Apple Inc., n.d.-b).
- **UserNotifications and AVFoundation:** alerts and custom alert sounds.
- **Python:** a log analyzer that recomputes the reported figures from the CSV logs.

Version Control

The source code, test logs and analysis scripts are published in a public GitHub repository, with a project website at https://hadik-uofa.github.io/TagAlert, as listed in the proposal: https://github.com/Hadik-UofA/TagAlert. The project used build letters (5a to 6b) shown in the app's Settings and written into every log, so each result can be tied to the exact code that produced it.

Changes from the Proposal

- **Android and Kotlin:** not built. The proposal marked Android as time-permitting; it remains future work.
- **XCTest:** no automated test target is included. Testing was done on the device against written protocols and logs.
- **MapKit:** not used. A map cannot show where a tag is, because Bluetooth gives no direction or reliable distance.
- **App Store submission:** not done. The app uses an undocumented AirTag service, and submission was outside the course scope.

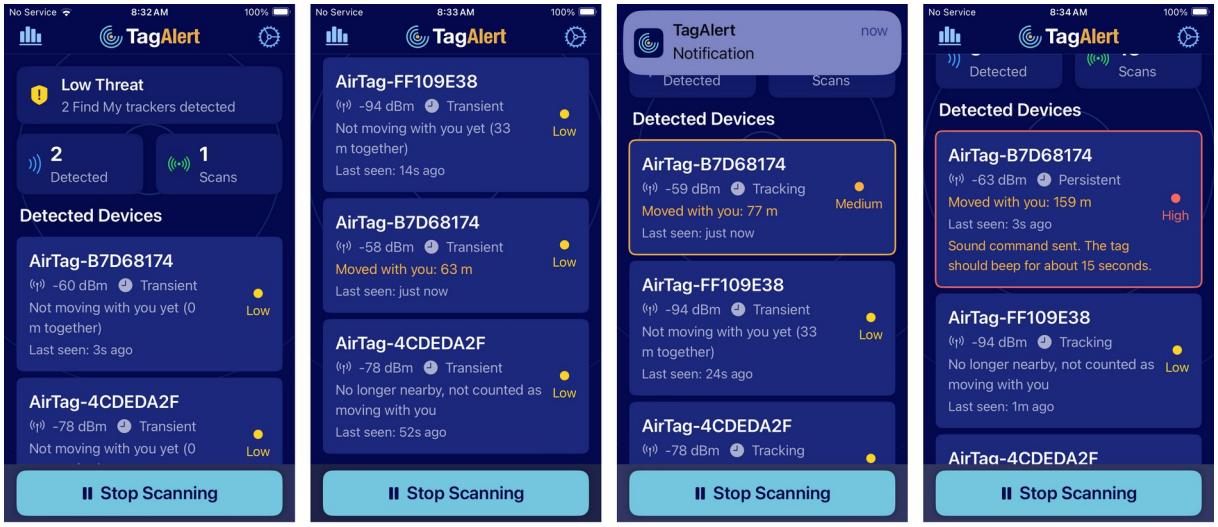
Results and Final Product

Application Features (Version 6 build b)

- **Live scanning dashboard:** detected AirTags, scan count, and a status card while scanning, with a radar animation in the background (still rings when iOS Reduce Motion is on).
- **Per-tag rows:** shortened identifier, signal strength, time seen, level, and a plain note such as "Moved with you: 73 m". Medium tags have an amber edge and High tags a red edge; red is used only for High.
- **Make This Tag Beep** and the one-time "Find This Tag?" prompt at High.
- **Safety and reporting guide:** steps to take, including Apple's instructions for removing the battery and when to contact authorities.
- **Session summary:** scan cycles, scan interval, current threat levels, and every tag detected in the session with how long it was seen.
- **Settings:** scan interval, Adaptive Scanning, movement requirement, notification levels, alert sound picker with preview, automatic beep (off by default), test mode, log export, and version.

Figure 1

Walking Test B7, Build 6b, November 9, 2026

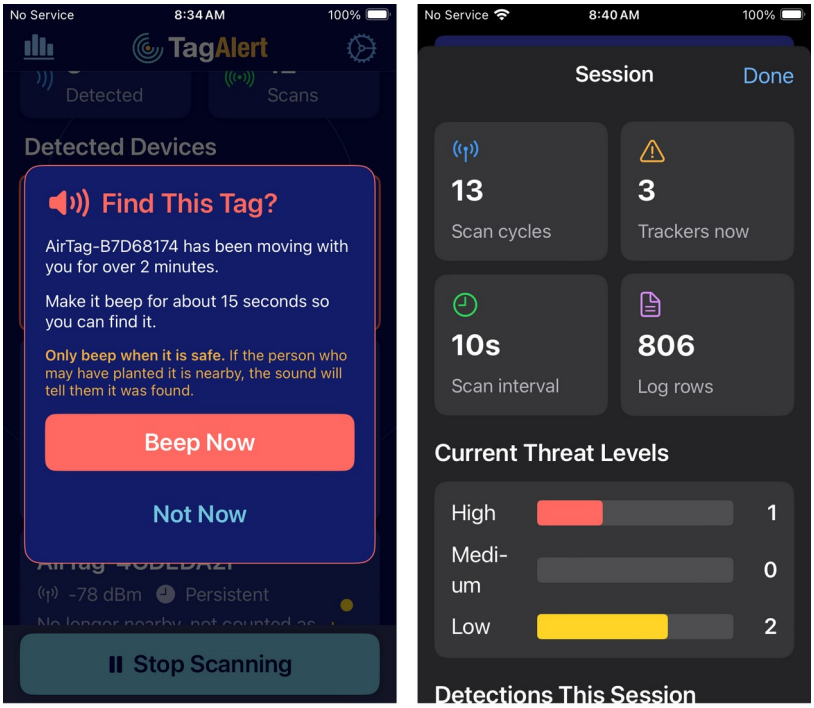


(a) 10 s *(b) 65 s* *(c) 75 s* *(d) 137 s*

Note. Log: log tagalert_log_20261109_082940.csv; times are seconds after Start Scanning. (a) The tag in the pocket is confirmed and shown at Low. (b) It has moved 63 m with the user but has been seen for under a minute, so it stays Low; the user's tag left at home shows "No longer nearby". (c) Medium at 77 m, with an amber edge and a notification. (d) High at 159 m, with a red edge, after the beep was sent.

Figure 2

Find This Tag? Prompt and Session Summary From Walk B7



(a) Find This Tag? *(b) Session summary*

Note. (a) The one-time Find This Tag? prompt when the tag reached High in B7, with its safety warning. (b) The session summary after B7: one tag at High, two at Low.

Detection Time

Table 1 shows the time from pressing Start Scanning to a confirmed AirTag, for tags present when scanning began.

Table 1

Time to Confirm an AirTag

Probe order	Builds	Confirmations	Range	Median
First-in, first-out	Nov 2 to 5n	24	6.9 to 136.5 s	63.0 s
Strongest signal first	5o to 6b	21	2.9 to 28.0 s	10.4 s

Note. Includes all logged confirmations of tags present at the start of a session. Not a controlled comparison: the scan interval also changed (30 s to 15 s in build 5q), and tags were closer to the phone in later stationary tests.

In the six walking tests run with strongest-first probing, the tag in the user's pocket was confirmed within 9 seconds in five; in B6 it was first heard 20 seconds into the walk and confirmed at 28 seconds. On the table, all three tags were confirmed within 15 seconds in A7, and in A8, with results shown immediately, they appeared on screen at 10.1, 11.6 and 13.1 s. In A8 the two strongest devices in the room, a computer and a phone, were probed first and were not AirTags. Every confirmed AirTag in the logs advertised without a transmit-power field, which most phones and computers include, so probing devices without that field first could save a few seconds (future work). A tag in another room (A5 and A6) was heard rarely, about once in 100 seconds in A6, and was not confirmed before those short sessions ended. Detection of a distant, stationary tag is slow; detection of a tag on the user is fast.

Walking Tests

Table 2

Walking Tests With One AirTag in the User's Pocket

Trial	Build	Confirmed	Medium	High	False Medium or High
B1	5n	94.1 s	171.5 s / 196 m	not reached (stopped at 184 s)	0
B2	5o	6.5 s	96.0 s / 117 m	not reached (stopped at 244 s)	1 (DB28A10D)
B3	5r	6.0 s	71.2 s / 67 m	not reached (stopped at 160 s)	1 (A261B482, 20 s)
B4	5s	8.9 s	61.8 s / 58 m	305.6 s / 380 m (5-minute rule)	0
B5	5t	7.1 s	71.4 s / 73 m	152.9 s / 179 m (2.5-minute rule)	0
B6	6a	28.0 s	86.9 s / 93 m	178.8 s / 158 m (2.5-minute rule)	0
B7	6b	6.1 s	71.6 s / 77 m	122.7 s / 148 m (2-minute rule)	0

Note. Times are seconds after Start Scanning; distances are walked distance from the step counter. A notification fired at every Medium and High shown. In B6 and B7 the Find This Tag? prompt appeared at High and Beep Now was accepted.

The first walk exposed a slow probe queue (fixed in 5o). The next two each produced one false Medium on a tag that was not the user's, each with a different root cause:

- **B2:** a parked tag near home was seen once early in the loop and again near the end. The rule counted the 169 m walked between the two sightings as moving together. Fix (5p): the tag must be present in at least 60% of scan cycles, and the window resets after 45 s unseen instead of 120 s.
- **B3:** a tag passed on the route kept accumulating distance from the step counter after it was last seen. It reached Medium for 20 seconds, then dropped back to Low. Fix (5s): distance counts only up to the tag's last sighting, and the tag must have been seen within the last 25 s. Applied to the B3 log, this tag would have reached 31 m, below the 50 m threshold.

With both fixes in place, B4 to B7 produced no false Medium or High. In B5 an unknown tag (B7FB4182) did travel with the user for about 51 m at 100% presence, but it had been seen for only 34 seconds, so the one-minute time rule held it at Low, and 30 seconds later it no longer counted as moving. High was then shortened twice. After B4 it moved from 5 minutes to 2.5, because High already requires movement and 380 m of walking was judged too long for the strongest warning; B5 confirmed High at 153 s and 179 m, matching the prediction from B4's log. After B6 it moved to 2 minutes. A shorter walking distance (25 m instead of 50 m) was considered instead and rejected: since build 5r the time rules, not the distance, have set when

alerts happen, so a shorter distance would not have made alerts sooner but would have let passed tags qualify more easily. B7 confirmed High at 123 s and 148 m.

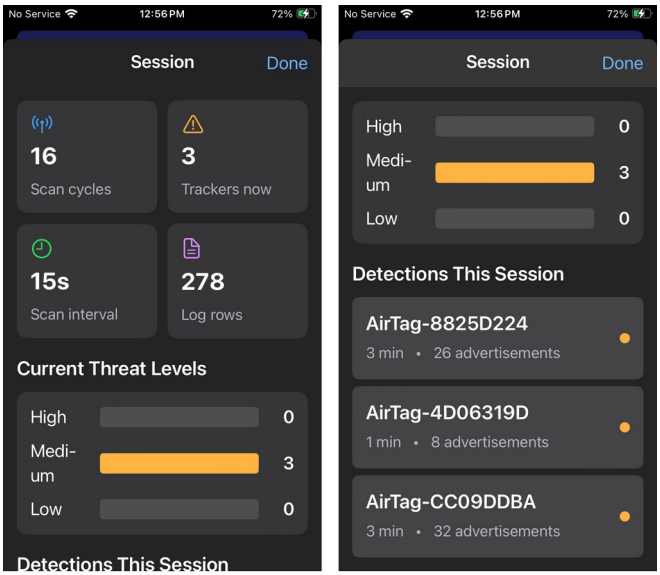
Unknown AirTags appeared on 4 of the 7 walks (DB28A10D, A261B482, B7FB4182 and FF109E38) along an ordinary residential route. This is an informal but useful indication of how often a detector will see other people's tags.

Stationary Control Tests

Before the movement check, every stationary tag reached Medium within 3 to 74 seconds of first being seen, producing 9 Medium notifications across three sessions on November 2 and 8 with no tag moving (Figure 3). Table 3 shows the same situation with the movement check on.

Figure 3

Stationary Tags Rated Medium Before the Movement Check



(a) Threat levels **(b) Detections**

Note. An earlier build on November 8 at 12:56 PM (log tagalert_log_20261108_124708.csv), with three AirTags at home all rated Medium after 1 to 3 minutes although none was traveling with the user.

Table 3

Stationary Controls With the Movement Check On

Test	Build	Length	Tags	Highest level	Notifications	What it showed
A3	5m	5.4 min	3	Low	0	601 tag-seconds held at Low that the old rules scored Medium
A4	5m	10.4 min	3	Low	0	All tags passed 5 minutes; one read -51 to -60 dBm, which the old rules scored High
A5	5w	1.3 min	3	Low	0	Distant-tag retry scheduled instead of written off
A6	5x	1.9 min	3	Low	0	Distant tag heard once in about 100 s
A7	5y	2.6 min	3	Low	0	All three confirmed within 15 s; beep test (below)
A8	6a	0.6 min	3	Low	0	Tags on screen at 10 to 13 s; beep 4 of 4
BAT1	6b	46.7 min	4	Low	0	Battery run; 274 scan cycles; first tag confirmed at 2.9 s

Note. Logs: 190309 (A3, A4), 210350 (A5), 211050 (A6), 212441 (A7), 20261109_080815 (A8), 20261109_084433_BAT1 (BAT1).

A 6-minute stationary test (A2, build 5l) also exposed a defect in the original threat rules: a tag seen for 5 minutes with signal weaker than -60 dBm matched no rule and fell to None, so four tags present longer scored lower than tags present for 2 minutes. This defect had been in the design since the start; it was fixed in 5m and A4 confirmed the fix.

Make This Tag Beep

Across all logs, 37 beep commands were sent and 26 were accepted. Every accepted command went to one of the user's own tags while separated from its owner. In the first beep test (log 141425), all three of the user's tags were heard and located, 4 commands of 4. The failures fall into three groups:

Table 4

Beep Commands, November 8 and 9, 2026

Tag group	Accepted / sent	Signal	Explanation
User's tags, separated from owner	26 / 27	-27 to -91 dBm	One timeout at -90 dBm (weak signal)
Known non-user tags	0 / 6	-91 to -98 dBm	Weak signal, likely near their owners
4D06319D (owner unknown)	0 / 1	-89 dBm	Weak signal; rotating addresses prevent matching it to a tag
4CDEDA2F, owner nearby	0 / 3	-40 to -59 dBm	Owner's phone in the house; Apple blocks the sound

Note. Includes all logs.

Key finding. Test A7 gave the first direct evidence for the beep's operating condition. On the same table, at strong signal, the two tags separated from their owner each beeped on the first try, while a tag whose owner's phone was in the house refused all three commands even though it accepted the detection connection within 2 seconds. The beep works in the planted-tag case, where the owner is away, and by Apple's design does not work when the owner is present. The next morning (A8), with the owner's phone away, the same tag accepted the beep on the first try: refused 3 of 3 with its owner nearby, accepted 1 of 1 with its owner away. The tag had also been running longer since a battery change by then, so a repeat that switches the owner's Bluetooth on and off in one session would rule that out. The limit cuts both ways: it also means a beep cannot alert a planter who is nearby with their phone.

Battery

Over 47 minutes of continuous foreground scanning with the screen on, TagAlert used 13% of an iPhone SE (2nd generation) battery, about 17% per hour. Screen-on use is included, so the app's own share is lower.

Table 5

Battery Run BAT1, November 9, 2026

Measure	Value
Build and conditions	6b; unplugged; live foreground scan; screen on (Auto-Lock off); Low Power Mode off; no other use
Scan time	46.7 minutes, 274 scan cycles (15 s interval, 10 s once a tag was found)
Status bar (1% steps)	100% at 8:45 AM to 87% at 9:34 AM: 13% used, 16.7% per hour
App log (iOS reports 5% steps to apps)	100% to 85%; crossed 95% at 23 min, 90% at 37 min, 85% at 46 min
Tags present	4, all Low for the whole run; no notifications

Note. Log tagalert_log_20261109_084433_BAT1.csv.

The two readings agree within the app log's 5% step. Caveats: this is a single run; it started straight off the charger at 100%, where the first few percent can drain slowly; and the screen was on throughout, so 17% per hour is an upper bound for TagAlert itself. The run also served as a seventh stationary control: three tags beside the phone and a fourth weak tag (-93 to -97 dBm, in another room or next door) stayed Low for 47 minutes with no notifications. The weak tag timed out on early probes and was confirmed about 17 minutes in, showing the retry back-off working on a distant tag.

Limitations and Constraints

1. **Foreground only.** Scanning stops when the app is in the background. iOS does not deliver the unfiltered scan results this method needs to a suspended app.
2. **Small test.** One iPhone model and iOS version, one home, one walking route, three to four AirTags, 21 log files over 9 days. Results show the method works; they are not accuracy rates.
3. **No GPS reference.** Walking distances are step-counter estimates.
4. **Vehicle path untested.** The rule counts 60 s in a vehicle as movement, but no car test was run.

5. **Movement needs Fitness Tracking.** If the user has turned it off, levels use time only and High is capped at Medium.
6. **Beep needs owner separation.** The beep works only on a tag separated from its owner and in range.
7. **Signal strength is not distance.** Two tags side by side read about 15 decibels (dB) apart.
8. **Distant tags are slow to confirm.** A tag behind walls is heard rarely and is slow to accept a connection.
9. **Cannot see whether a tag is with its owner.** AirTags broadcast whether they are near their owner or separated, but iOS withholds that broadcast from third-party apps. A stranger's tag traveling with its owner (for example, a fellow commuter's keys) is not a tracking threat, yet it is flagged like a planted one if it stays with the user long enough.
10. **AirTags only.** Other Find My accessories, Samsung SmartTags and Tile trackers are not detected.
11. **One battery run.** About 17% per hour with the screen on, from a single 47-minute run; screen-off and longer runs were not measured.

Discussion and Analysis

Answers to the Research Questions

- **Battery Optimization:** the design limits radio time with short 5-second scan windows, an adjustable interval, and an adaptive mode that lengthens the interval on low battery. Walk tests led to shorter intervals (15 s, then 10 s once a tag is found) for faster response, which roughly doubles radio-on time. The measured cost: about 17% of the phone's battery per hour with the screen on (Table 5), which suits a check of minutes to an hour rather than all-day monitoring. Because TagAlert scans only while it is open, the user

also controls this cost: running a check when it is needed, rather than leaving the app open all day, limits the drain to that period.

- **User Experience Design:** feedback during testing removed clutter (a status card shown only while scanning), reserved red for High, replaced default notification sounds that were too quiet, and explained permission problems in plain steps. No formal usability study with outside users was run; this is future work.
- **Multiple Device Detection:** each tag is tracked, scored and notified independently. The app handled three of the user's tags plus unknown tags in the same session, and up to about 270 queued bare devices on a walk.
- **False Positive Reduction:** the strongest result of the project. Time alone produced Medium alerts on every stationary tag. Requiring movement, presence across scans and a recent sighting removed stationary alerts in seven control tests and passed-tag alerts on the last four walks.
- **Threat Assessment:** time plus movement, without signal strength. Medium at 1 minute and High at 2 minutes of shared movement.

Technical Challenges and Solutions

Challenge 1: iOS hides the AirTag advertisement

The first live scans found no AirTags at all. The working hypothesis was that iOS withholds Apple's advertisement data, but no published source confirmed it, so it was tested against a working reference: AirGuard on the same phone found the tag, its source showed the connection-based method, and TagAlert's reimplementation confirmed three AirTags on November 2. The approach was to settle the question empirically rather than from documentation.

Challenge 2: Signal strength is not a reliable guide to distance

The original rules required signal stronger than -60 dBm for High. In test A4, two tags inches from the phone read -51 to -60 dBm and -66 to -76 dBm, about 15 dB apart, and the second never reached the High threshold. Likely causes are antenna orientation, the body or phone blocking the signal, and differences between tags; the tests did not separate them. Signal strength was removed from the rules in 5n.

Challenge 3: The walking route produced its own false positives

A loop route passes the same parked tags near the start and the end, and a step counter keeps counting after a passed tag is gone. Both produced false Medium alerts that a stationary test could never reveal. Each was fixed with a specific rule (presence, recent sighting) and retested on the same route.

Challenge 4: Finding a tag that should have been easy

One tag "consistently" failed to appear. The logs showed it was weak because it was in another room, and that two timeouts had written it off for the session. The fix was in the queue (retries and a queue-age rule), not the tag. A further tag that never appeared in any log turned out to have no battery: a dead AirTag broadcasts nothing.

Choosing the Alert Times

High was tested at 5 minutes, 2.5 minutes and 2 minutes, with no false alerts at any of them on the test route. The faster the warning, the sooner a person learns a tag is traveling with them, so the question is how short the times can be before strangers' tags start to qualify. On the test route the longest any unknown tag counted as moving with the user under the final rules was about 30 seconds (B7FB4182 in B5), and the 1-minute Medium rule held it at Low. On this evidence the binding limit is Medium, not High, and the data cannot yet tell 2 minutes from 1.5.

1.5 minutes was not adopted, because counting a tag as moving takes at least four scan cycles (about 40 seconds) plus 50 m of walking, which would leave Medium a window of only 30 seconds. The final choice should come from walks on varied routes (busy sidewalks, transit, shared commutes) that measure how long strangers' tags stay with a person. The logs already record this.

Tags Traveling With Their Owners

A stranger's AirTag riding the bus with its owner is not tracking the user; it reports its owner's location. Apple's own alerts are built on this distinction and warn only about a tag that is separated from its owner. The tag's broadcast says which state it is in: the Internet Engineering Task Force (IETF) draft for detecting unwanted location trackers, written by Apple and Google, defines a near-owner bit (Ledvina et al., 2026). iOS does not pass that broadcast to third-party apps, which is why TagAlert must connect to each device. A commuter's tag therefore looks the same as a planted one. Such false alerts cost the user's trust rather than their safety, while a slow warning about a planted tag costs safety, which supports keeping the High time short.

Two cautions apply. A tag near its owner now can be a threat later: a tag planted in a car by a partner who rides along reads as near its owner until they leave. And a test beep cannot safely be used to sort tags, because a truly planted tag would sound and could alert the planter. The beep refusal in Finding 20 is the only owner-state signal available on iOS today.

Privacy and Ethical Considerations

- **No location.** Movement comes from the step counter and activity type. TagAlert has no account and no server, and sends nothing off the phone.
- **Respecting the user's privacy settings.** TagAlert does not work around Fitness Tracking being off; it explains the setting and degrades to time-only levels.

- **Beep is opt-in at High.** Domestic-violence safety guidance warns that discovering a tracker can raise the risk from the person who planted it (National Network to End Domestic Violence, Safety Net Project, n.d.). A beep could tell a nearby abuser the tag was found, so the app asks once and leaves automatic beeping off by default. Automatic beeping would also sound strangers' tags that happen to travel with the user.
- **Careful wording.** A refused beep at strong signal may mean the owner is nearby, but other causes produce the same timeout. TagAlert says only "Timed out. The tag may be near its owner; out of range; or busy." rather than presenting a guess as a safety signal.
- **Third parties.** TagAlert detects other people's tags but stores only shortened, rotating identifiers in a log that stays on the phone unless exported.

Comparative Analysis with Existing Solutions

Apple's built-in protection warns iPhone owners about unknown trackers and makes separated AirTags play a sound after 8 to 24 hours. TagAlert adds an on-demand scan that the user controls, warnings after about a minute of shared movement in testing, and a way to make the tag beep. It does not replace Apple's system, which works in the background and TagAlert does not.

AirGuard is the research reference for the detection method and covers more tracker types and platforms. It is built for background protection, which operating systems deliberately limit: on Android its background scans run at most every 15 minutes, and it warns after seeing a tracker in at least three different locations (Heinrich et al., 2022). TagAlert covers the other need: an immediate answer. Opened on demand, it shows nearby AirTags within seconds and whether one is moving with the user within one to two minutes, without using location. Its other

contributions are a movement-based threat model tuned with field tests and documented failures, a safety-first beep design, and a public test log for every number reported.

Skills and Competencies Gained

Technical Skills Development

- **Bluetooth Low Energy:** advertisement structure, Generic Attribute Profile (GATT) services and characteristics (the structure a device uses to publish its features), connection timing, and the practical behavior of signal strength.
- **iOS development:** Swift, SwiftUI, Core Bluetooth, Core Motion, notifications, and on-device deployment.
- **Reverse-engineering literacy:** reading a research group's open-source code and papers to understand an undocumented protocol, and reimplementing the method independently with attribution.
- **Test design and analysis:** control tests, markers in logs, root-cause analysis, and rule changes tied to evidence.

Cybersecurity Application

- Threat modeling for a physical tracking threat, including the abuser-nearby case.
- Privacy by design: no location, no server, no workaround of user privacy settings.
- Evidence-based security claims: every reported figure traceable to a test log.

Project Management Experience

- Staged delivery: a simulated-data prototype to validate the interface, then a live build to validate detection in the field.
- Version discipline: a build letter for every change, logged with every result.

- Scope control: focusing the live build on detection, threat assessment and locating the tag, and deferring maps and background scanning.

Conclusion and Future Directions

Project Summary

TagAlert detects AirTags on an iPhone even though iOS hides the data that identifies them, warns when a tag travels with the user rather than when one is merely nearby, and helps the user find it. In the final test, a tag in a pocket was confirmed in 6 seconds and raised High after 123 seconds and 148 meters of walking, while stationary tags raised no alerts. The test set is small, and the results are reported with that limit.

Significance and Impact

The project shows that a third-party iPhone app can give a person an on-demand check for a planted AirTag, and it documents in detail where such an app fails: background operation, distant tags, tags near their owner, and strangers' tags that really do travel alongside. Those failures matter as much as the successes for anyone building or relying on such a tool.

Future Development Opportunities

1. **Owner-nearby signal:** a beep refused at strong signal may indicate the owner is nearby, which matters for safety. Before showing this to users it needs controlled trials separating it from other timeout causes, a measured false-alarm rate, and safety-reviewed wording.
2. **Reading the owner state:** an Android version (Android apps can read the raw broadcast) or a small Bluetooth companion device (for example an ESP32 or nRF52840 board) relaying the near-owner bit to the iPhone would let TagAlert set aside tags traveling with their owners. The standard's non-owner connection interface may also expose it; this is unverified.

3. **Alert-time study:** walks on varied routes to measure how long strangers' tags stay with a person, then set Medium and High from that false-alert rate.
4. **Background protection:** not possible with this method on iOS today; would require a different approach or platform support.
5. **Critical Alerts:** an Apple entitlement that lets alerts sound in silent mode, granted to safety apps.
6. **Vehicle and GPS reference tests:** test the 60-second vehicle rule and measure step-counter accuracy against GPS.
7. **Larger study:** more phones, iOS versions, locations and participants, including a usability study with non-technical users.
8. **Other trackers and Android:** Find My accessories, Samsung SmartTag, Tile, and an Android version as proposed.

Lessons Learned

Simulate first, then measure. The simulated-data prototype made it possible to design and test the interface quickly. Field testing then produced the numbers, and every rule in the final app exists because a specific log showed the previous rule could be improved.

Test where the product will be used. Stationary tests made the threat rules look finished. The first walk on a real street exposed a slow queue and two kinds of false alert that no table-top test could have shown.

Safety features can be unsafe. Automatically beeping a tag sounds helpful until the person who planted it is in the same room. Designing for the worst realistic user situation changed the default.

Personal Growth and Professional Development

Taking the project from a simulated prototype to a live, field-tested app, and reporting every result with its evidence and its limits, is the central achievement of this capstone. It is also the standard the author will carry into future security research.

References

Apple Inc. (n.d.-a). *Core Bluetooth*. Apple Developer Documentation. https://developer.apple.com/documentation/corebluetooth

Apple Inc. (n.d.-b). *Core Motion*. Apple Developer Documentation. https://developer.apple.com/documentation/coremotion

Heinrich, A., Bittner, N., & Hollick, M. (2022). AirGuard: Protecting Android users from stalking attacks by Apple Find My devices. In *Proceedings of the 15th ACM Conference on Security and Privacy in Wireless and Mobile Networks* (pp. 26–38). Association for Computing Machinery. https://doi.org/10.1145/3507657.3528546

Heinrich, A., Stute, M., Kornhuber, T., & Hollick, M. (2021). Who can find my devices? Security and privacy of Apple's crowd-sourced Bluetooth location tracking system. *Proceedings on Privacy Enhancing Technologies, 2021*(3), 227–245. https://doi.org/10.2478/popets-2021-0045

Ledvina, B., Lazarov, D., Detwiler, B., & Polatkan, S. P. (2026). *Detecting unwanted location trackers accessory protocol* (Internet-Draft draft-ietf-dult-accessory-protocol-01). Internet Engineering Task Force. https://datatracker.ietf.org/doc/draft-ietf-dult-accessory-protocol/

MacTech. (2021, June 3). *Apple is beefing up its AirTags privacy features*. https://www.mactech.com/2021/06/03/apple-is-beefing-up-its-airtags-privacy-features/

National Network to End Domestic Violence, Safety Net Project. (n.d.). *Technology safety*. https://www.techsafety.org

Secure Mobile Networking Lab. (n.d.). *AirGuard-iOS* [Computer software]. GitHub. https://github.com/seemoo-lab/AirGuard-iOS

Appendices

Appendix A: Technical Specifications

- **Tested on:** iPhone SE (2nd generation), iOS 17.6.1.
- **Permissions:** Bluetooth (required), Motion & Fitness (for the movement check), Notifications.
- **Network:** none required. No data leaves the phone unless the user exports the log.
- **Source size:** about 2,300 lines of Swift in five files.
- **Final build:** Version 6 build b.

Appendix B: Rule Changes Driven by Testing

Build	Change	Evidence
5a to 5g	Movement check via Core Motion (no location); Fitness Tracking prompt and fallback	Stationary tags reached Medium in 3 to 74 s (Nov 2 and Nov 8 logs)
5m	Tag can no longer drop to None after 5 minutes	A2: four tags went Low to None at 300 s
5n	Levels use time and movement only; signal removed	A4: tags side by side about 15 dB apart
5o	Strongest signal probed first; time counts from first sighting	B1: pocket tag waited 94 s in the queue
5p	Presence in 60% of scans over 4 cycles; reset after 45 s unseen	B2: parked tag passed twice on a loop
5q, 5r	Scan every 15 s (10 s once a tag is found); walk threshold 50 m	Responsiveness after B2
5s	Distance counts only to last sighting; seen within 25 s	B3: passed tag gained distance after it was gone
5t	High at 2.5 minutes; red only for High	B4: 380 m of walking before High
5u	"Find This Tag?" prompt at High; auto-beep off by default	Safety review of auto-beep
5w, 5x	Retries with back-off; queue-age rule; longer timeout for weak devices	B4, B5, A5: distant tag written off or starved
5y, 5z	Radar Pulse default alert sound; Safety Guide colors	User feedback
6a	Probing starts about 1 s into a scan; confirmed tags shown at once	A7: first tag confirmed at 11 s, shown at 20 s
6b	High at 2 minutes (distance kept at 50 m)	B6: time rules, not distance, set alert times

Appendix C: Test Log Index

All logs are CSV files named tagalert_log_YYYYMMDD_HHMMSS.csv (local start time) and are published with the source code.

Log	Test	Build	Purpose
20261101_222156, 225048	First live scans	v2	Zero Apple manufacturer data with an AirTag nearby
20261101_231617	FD44 filter	v2	Cached false detections (filter removed)
20261102_080459	First detection	v2 + probe	Three AirTags confirmed by connection
20261108_124708, 141425	Beep and detection	pre-5h	Beeps 4 of 4 on own tags; stationary tags at Medium
20261108_180041	Detection	pre-5h	Movement check, stationary
20261108_184851	A2	5l	None-level defect at 300 s
20261108_190309	A3, A4	5m	Stationary controls
20261108_193256	B1	5n	Walk: slow queue
20261108_194852	B2	5o	Walk: parked-tag false Medium
20261108_200330	B3	5r	Walk: passed-tag false Medium
20261108_201609	B4	5s	Walk: Low, Medium, High; no false alerts
20261108_203337	B5	5t	Walk: High at 2.5 minutes; no false alerts
20261108_210350	A5	5w	Stationary; distant-tag retry
20261108_211050	A6	5x	Stationary; distant tag heard rarely
20261108_212441	A7	5y	Stationary; owner-nearby beep finding
20261109_080815	A8	6a	Tags shown on confirmation; beep with owner away
20261109_080815_2	A8, B6	6a	Export containing A8 and walk B6
20261109_082940	B7	6b	Walk: High at 2 minutes; Find This Tag? prompt and beep
20261109_084433_BAT1	BAT1	6b	47-minute battery run; stationary control

Appendix D: User Guide Quick Reference

1. Open TagAlert and allow Bluetooth, Motion & Fitness, and Notifications.
2. If a banner says Fitness Tracking is off, follow its steps to turn it on in iPhone Settings.
3. Tap Start Scanning and keep the app open.
4. **Low:** nothing to do; most nearby tags are other people's and are not moving with the user.

5. **Medium:** a tag has traveled with the user for about a minute. Check bags, pockets and the car.
6. **High:** a tag has traveled with the user for 2 minutes or more. If it is safe, tap Beep Now to find it, then follow the safety guide.

Appendix E: Troubleshooting

- **No tags found:** keep the app open and the tag within a few meters. A tag near its owner, behind walls, or with a dead battery may not appear or may take longer.
- **Levels never go above Low while walking:** Fitness Tracking may be off (see the banner), or the tag is not being seen in most scans.
- **Beep times out:** the tag may be near its owner, out of range, or busy. Move closer and try again.
- **Alert is silent:** the ring switch or a Focus mode mutes alert sounds.

Acknowledgments

Thanks to the University of Arizona faculty for the instruction that made this project possible and for the opportunity to complete it. The detection method builds on published research and open-source code from the Secure Mobile Networking Lab at TU Darmstadt (AirGuard); their decision to publish both made this independent implementation possible.

Project Completion Date: November 9, 2026

Final Submission: APCV 498 Capstone Project

Student: Elizabeth Hadik-Barkoczy

Project Duration: Fall 2024 (September 30 to December 2024); rebuild and field testing November 1 to 9, 2026